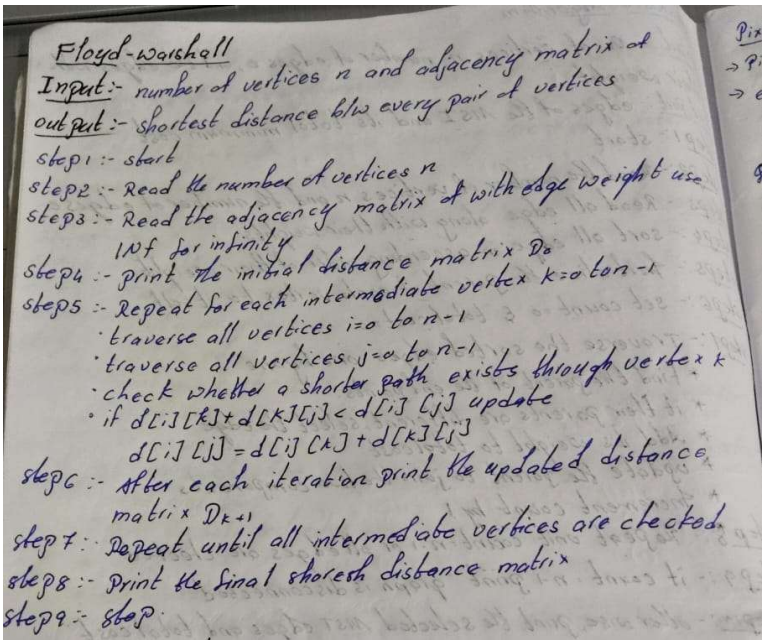
PRACTICAL: 11

AIM:- Write a program for Floyd-Warshal algorithm.

Algorithms:-



Program/code:-

```
#include <iostream>
using namespace std;
#define INF 99999
int main()
{
    int n, d[10][10];
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter adjacency matrix (99999 for infinity):\n";
```

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        cin >> d[i][j];
cout << "\nD0 Matrix:\n";
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++)
        cout << d[i][j] << " ";
    cout << endl;
}
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (d[i][k] != INF && d[k][j] != INF &&
                d[i][k] + d[k][j] < d[i][j]) {
                d[i][j] = d[i][k] + d[k][j];
            }
        }
    }
    cout << "\nD" << k + 1 << " Matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (d[i][j] == INF)
                cout << "INF ";
            else
                cout << d[i][j] << " ";
        }
        cout << endl;
```

```
    }
  }
  return 0;
}
```

Output:-

```
input
Enter number of vertices: 4
Enter adjacency matrix (99999 for infinity):
0 9 -4 99999
6 0 99999 2
99999 5 0 99999
99999 99999 1 0

D0 Matrix:
0 9 -4 99999
6 0 99999 2
99999 5 0 99999
99999 99999 1 0

D1 Matrix:
0 9 -4 INF
6 0 2 2
INF 5 0 INF
INF INF 1 0

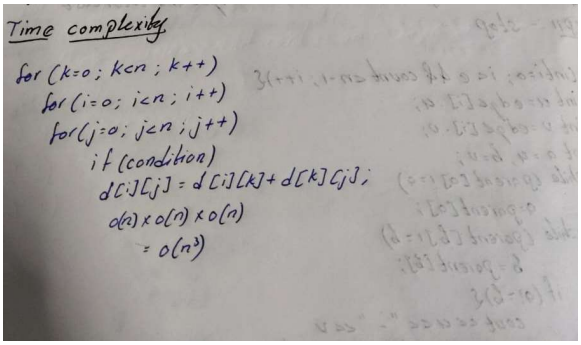
D2 Matrix:
0 9 -4 11
6 0 2 2
11 5 0 7
INF INF 1 0

D3 Matrix:
0 1 -4 3
6 0 2 2
11 5 0 7
12 6 1 0

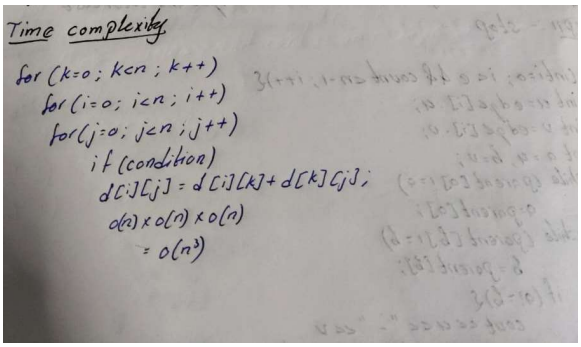
D4 Matrix:
0 1 -4 3
6 0 2 2
11 5 0 7
12 6 1 0
```

Time complexity:-

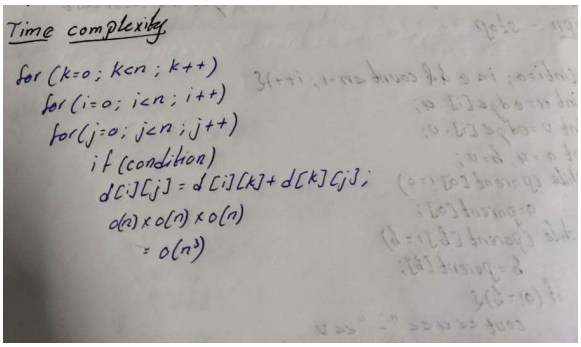
Best case:-



Worst case:-



Average case:-



Conclusion :-

The Floyd-Warshall algorithm is a dynamic programming approach used to find the shortest paths between all pairs of vertices in a weighted graph. It works for both directed and undirected graphs and handles negative edge weights, provided there are no negative weight cycles.